

PostgreSQL Blog

★ Member-only story

Why The Guardian Ditched MongoDB for PostgreSQL: Behind the Great Database Migration



Oz

Following ▾

4 min read · Jun 26, 2026



176



2



How one of the world's largest news organizations woke up from the NoSQL hype and went back to a traditional relational database. Here's the story — and the massive lesson it holds for the tech world.



Tech trends move fast. There was a time when everyone wanted to force microservices into every single project, and another era when NoSQL databases were aggressively marketed as the ultimate cure for every backend

headache. The UK media giant **The Guardian** was one of the early adopters who caught this wave. When building their custom Content Management System (CMS), *Composer*, they decided to roll with MongoDB.

But years later, they made a radical move: they migrated all that data to PostgreSQL.

So, what happened? Was MongoDB a bad database? Absolutely not. The real issue was much simpler: they were using the wrong tool for the job. Let's break down the behind-the-scenes of this massive migration.

The MongoDB Honeymoon Phase

When The Guardian first started architecting their new CMS, they desperately needed flexibility. A news article isn't just text; it contains videos, photos, author bios, metadata, and ever-changing tags. Back in the early 2010s, MongoDB's schema-less nature and JSON-like document model seemed like a match made in heaven.

Developers could write code at lightning speed without having to wrestle with rigid database schemas. In the beginning, everything felt like a dream.

The Reality Check: The Data Was Actually Relational!

As the system scaled, The Guardian's engineering team hit a wall and realized a fundamental truth: **their data was highly relational**.

Think about how a news article works. An article belongs to a specific author, falls under certain categories, and is linked to multiple tags. Because MongoDB is a document-oriented database, managing relational data (what we know as JOINS in the SQL world) goes against its very nature.

Whenever they needed to display an article along with all its associated metadata, they couldn't just fetch it in one go. They had to pull fragmented pieces of data from the database and stitch them together behind the scenes.

The Nightmare of Application-Level JOINS

Because you couldn't easily join tables in MongoDB, The Guardian's developers had to handle these JOINS at the application level. This forced approach led to three massive headaches:

- **Bloated Codebase:** Developers were writing complex, custom code to do the heavy lifting that a database should natively handle. The system became clunky.

- **Performance Hits:** Fetching all the related data required firing multiple separate requests to the database, slowing things down.
- **Data Integrity Risks:** Without the strict *Foreign Key* constraints of a relational database, there was always a lingering risk of orphaned data (like keeping an article tied to an author profile that had already been deleted).

Enter PostgreSQL (and the JSONB Game-Changer)

The team finally decided enough was enough. The solution was to pivot back to a traditional relational database, and they chose **PostgreSQL** — one of the most battle-tested, open-source RDBMS in the industry.

But here is where a beautiful coincidence (or rather, technological evolution) happened.

PostgreSQL hadn't just mastered relational data; it had recently introduced the **JSONB** data type. This was an absolute game-changer. It meant The Guardian could have the strict, rock-solid relational structure for their JOINS, *while still* keeping the flexible, document-style storage for unpredictable fields like article content. It was literally the best of both worlds.

The Aftermath of the Migration

Once The Guardian successfully ported their data over to PostgreSQL, the results spoke for themselves:

1. **A Leaner Codebase:** All that messy, application-level JOIN code was deleted. The heavy lifting was handed back to the database, right where it
2. **Bulletproof Data Integrity:** Thanks to PostgreSQL's ACID compliance and Foreign Key constraints, the reliability of their data was guaranteed.
3. **Boosted Performance:** Complex data relationships could now be fetched in milliseconds with a single, elegant SQL query.

The Takeaway

The Guardian's journey is a perfect reminder of one of software engineering's oldest rules: **There are no silver bullets.**

MongoDB is an incredibly powerful tool. If you are dealing with log management, IoT data streams, or real-time analytics, it will work wonders. But if your data has a fundamentally relational nature, forcing a NoSQL

database into the mix just because it's the trendy thing to do — will only build up a mountain of technical debt.

At the end of the day, the best technology isn't the one trending on X (Twitter). It's the one that solves your specific problem in the simplest, most reliable way possible.

Technology

Software Development

Software Engineering

Postgresql

The Guardian



Published in PostgreSQL Blog

Following

338 followers · Last published 9 hours ago

The technical resource for PostgreSQL internals, database design, and performance tuning



Written by Oz

Following ▾

2.4K followers · 20 following

Database Engineer 🐘

Responses (2)

🛡️



Josef Machytka he/him

What are your thoughts?



Sanjay Sinha

3 days ago

The UK media giant The Guardian

What media giant? It is just a newspaper and a utube channel.
Please correct me if I am wrong.

👏

💬 1 reply

Reply



Sasha Kacanski

4 days ago

gardian, a trash journalism ain't going to be helped by using postgres. would not read that trash of journalism if they pay me in gold. what a joke.

  1 reply [Reply](#)



